

---

# CS 329E Data Analytics Term Project

By: Sonia Siddiqui

# Steps

---

**01**

Cleaning

**02**

Clustering

**03**

Preprocessing

**04**

Dim reduction

**05**

Gaussian Mixture Model

**06**

Cluster Summary and  
Interpretation

**07**

Visualize Clusters in 2D

**08**

Save and Export



# Libraries:

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.mixture import GaussianMixture
from sklearn.metrics import silhouette_score
from sklearn.impute import SimpleImputer

sns.set(style="whitegrid", context="notebook")
```

**sklearn.preprocessing:** StandardScaler, OneHotEncoder. - Normalize numerical features and encode categorical variables.

**sklearn.impute:** SimpleImputer - Fill missing values in the dataset

**sklearn.compose:** ColumnTransformer - apply different transformations to numeric vs. categorical columns.

**sklearn.pipeline:** Pipeline - Chain steps (imputation → scaling → modeling) so data flows cleanly.

**sklearn.decomposition:** PCA - Reduce dataset complexity, speed up modeling, and visualize high-dimensional data.

**sklearn.manifold:** TSNE - Visualize high-dimensional data in 2D/3D, especially clustering

**sklearn.mixture:** GaussianMixture - Probabilistic clustering, soft cluster membership

**sklearn.metrics:** silhouette\_score - Measures cluster quality (higher = better separation)

# Dataset:

```
loans_df = pd.read_csv("NY_Loans_Purchased.csv")
loans_df
```

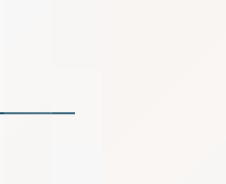
[25] Python

|       | Bond Series    | Original Loan Amount | Loan Purchase Date | Purchase Year | Original Loan To Value | Loan Type    | SONYMA DPAL/CCAL Amount | Original Term | County     | FIPS Code | Number of Units | Property Type | Housing Type | Household Size |
|-------|----------------|----------------------|--------------------|---------------|------------------------|--------------|-------------------------|---------------|------------|-----------|-----------------|---------------|--------------|----------------|
| 0     | HMB Warehouse  | \$117,370            | 06/30/2016         | 2016          | 97%                    | Conventional | \$3,630                 | 360           | Schuyler   | 36097     | 1 Family        | Detached      | Existing     | 1              |
| 1     | HMB Warehouse  | \$83,000             | 06/30/2016         | 2016          | 89%                    | Conventional | \$3,000                 | 360           | Chautauqua | 36013     | 1 Family        | Detached      | Existing     | 1              |
| 2     | HMB Warehouse  | \$133,860            | 06/30/2016         | 2016          | 97%                    | Conventional | \$0                     | 360           | Livingston | 36051     | 1 Family        | Detached      | Existing     | 2              |
| 3     | HMB Warehouse  | \$127,300            | 06/30/2016         | 2016          | 95%                    | Conventional | \$0                     | 360           | Erie       | 36029     | 1 Family        | Detached      | Existing     | 1              |
| 4     | HMB Warehouse  | \$74,150             | 06/30/2016         | 2016          | 78%                    | Conventional | \$0                     | 360           | Rensselaer | 36083     | 1 Family        | Detached      | Existing     | 3              |
| ...   | ...            | ...                  | ...                | ...           | ...                    | ...          | ...                     | ...           | ...        | ...       | ...             | ...           | ...          | ...            |
| 28523 | Series 109/110 | \$33,250             | 01/02/2004         | 2004          | 95%                    | Conventional | \$4,549                 | 360           | Oswego     | 36075     | 1 Family        | Detached      | Existing     | 1              |
| 28524 | Series 109/110 | \$101,000            | 01/02/2004         | 2004          | 96.19%                 | Conventional | NaN                     | 360           | Rensselaer | 36083     | 2 Family        | Detached      | Existing     | 1              |
| 28525 | Series 109/110 | \$80,000             | 01/02/2004         | 2004          | 96.5%                  | Conventional | \$4,218                 | 360           | Erie       | 36029     | 2 Family        | Detached      | Existing     | 1              |
| 28526 | Series 109/110 | \$52,300             | 01/02/2004         | 2004          | 96.85%                 | Conventional | \$4,383                 | 360           | Monroe     | 36055     | 1 Family        | Detached      | Existing     | 1              |
| 28527 | Series 109/110 | \$75,100             | 01/02/2004         | 2004          | 96.9%                  | Conventional | \$4,468                 | 360           | Broome     | 36007     | 1 Family        | Detached      | Existing     | 3              |

28528 rows x 14 columns

My dataset shows financial and geographic information on SONYMA Loans purchased since 2004 in New York. The data is provided by New York State Homes & Community Renewal. It shows loans taken out for mortgage payments.

---



**01**

# **Cleaning**

---



# Cleaning pt 1:

Convert currency and percentage text into numbers, detect dates, and standardize formats.

```
def parse_currency_column(s):  
    """Parse currency string like '$117,370.00' or '$3,630.00' to float. Returns NaN if cannot parse."""  
    if pd.isna(s):  
        return np.nan  
    try:  
        # remove $ and commas  
        return float(str(s).replace('$', '').replace(',', '').strip())  
    except:  
        return np.nan  
  
def parse_percent_column(s):  
    """Parse percent strings like '97%' or '97.0%' or numeric strings."""  
    if pd.isna(s):  
        return np.nan  
    try:  
        ss = str(s).strip().replace('%', '')  
        return float(ss)/100.0  
    except:  
        try:  
            return float(s)  
        except:
```

Currency stored as text cannot be used for math or machine learning. We convert them to numbers so models can interpret them.

# Cleaning pt 2:

```
# Find columns and convert them automatically
loans_df_clean = loans_df.copy()

# Clean up column names
loans_df_clean.columns = loans_df_clean.columns.str.strip()

# Map common name patterns -> conversions
for col in loans_df_clean.columns:
    lc = col.lower()
    ...# Currency detection by example values (if dtype object & contains '$')
    ...sample = loans_df_clean[col].dropna().astype(str).head(10).tolist()
    ...if any('$' in s for s in sample):
    ...    print("Parsing currency column:", col)
    ...    loans_df_clean[col] = loans_df_clean[col].map(parse_currency_column)
    ...# Percent detection
    ...elif any('%' in s for s in sample):
    ...    print("Parsing percent column:", col)
    ...    loans_df_clean[col] = loans_df_clean[col].map(parse_percent_column)
    ...# Year or date columns: try to convert Purchase Date-like columns to datetime
    ...if 'date' in lc or 'purchase' in lc or 'year' in lc:
    ...    # attempt datetime conversion
    ...    try:
    ...        loans_df_clean[col] = pd.to_datetime(loans_df_clean[col], errors='coerce')
    ...        if loans_df_clean[col].notnull().sum() > 0:
    ...            print("Parsed datetime column:", col)
    ...    except:
    ...        pass

# Show types after attempted conversions
display(loans_df_clean.dtypes)
```

Parsing currency column: Original Loan Amount  
Parsed datetime column: Loan Purchase Date  
Parsed datetime column: Purchase Year  
Parsing percent column: Original Loan To Value  
Parsing currency column: SONYMA DPAL/CCAL Amount

|                         |                |
|-------------------------|----------------|
| Bond Series             | object         |
| Original Loan Amount    | float64        |
| Loan Purchase Date      | datetime64[ns] |
| Purchase Year           | datetime64[ns] |
| Original Loan To Value  | float64        |
| Loan Type               | object         |
| SONYMA DPAL/CCAL Amount | float64        |
| Original Term           | int64          |
| County                  | object         |
| FIPS Code               | int64          |
| Number of Units         | object         |
| Property Type           | object         |
| Housing Type            | object         |
| Household Size          | int64          |
| dtype:                  | object         |

# Cleaning pt 3:

---



```
# Ensure all column names are stripped in your lists too
possible_num = [c.strip() for c in [
    ... 'Original Loan Amount', 'Loan Type', 'Loan Purchase Date', 'Original Loan To Value',
    ... 'Original Term', 'Purchase Year', 'FIPS Code', 'Household Size',
    ... 'SONYMA DPAL/CCAL Amount', 'Number of Units', 'Original Term'
]]

possible_cat = [c.strip() for c in [
    ... 'Loan Type', 'Property Type', 'Housing Type', 'County'
]]

num_cols = [c for c in possible_num if c in loans_df_clean.columns]
cat_cols = [c for c in possible_cat if c in loans_df_clean.columns]

print("Numeric columns used:", num_cols)
print("Categorical columns used:", cat_cols)
```

Python

```
Numeric columns used: ['Original Loan Amount', 'Loan Type', 'Loan Purchase Date', 'Original Loan To Value', 'Original Term', 'Purchase Year', 'FIPS Code',
Categorical columns used: ['Loan Type', 'Property Type', 'Housing Type', 'County']
```



---



**02**

# Clustering

---



# Select features for clustering:

```
# 1. Clean up any trailing spaces
loans_df_clean.columns = loans_df_clean.columns.str.strip()

# 2. Identify numeric, categorical, and datetime columns dynamically
num_cols = loans_df_clean.select_dtypes(include=['int64', 'float64']).columns.tolist()
cat_cols = loans_df_clean.select_dtypes(include=['object', 'category']).columns.tolist()

# 3. Convert datetime to numeric (ordinal form)
for col in loans_df_clean.select_dtypes(include=['datetime64']).columns:
    loans_df_clean[col] = loans_df_clean[col].map(lambda x: x.toordinal() if pd.notnull(x) else np.nan)
    num_cols.append(col)

print("Numeric columns used:", num_cols)
print("Categorical columns used:", cat_cols)

# 4. Drop rows missing all numeric data
df_subset = loans_df_clean[num_cols + cat_cols].copy()
df_subset = df_subset.dropna(subset=num_cols, how='all')
print("After dropna subset shape:", df_subset.shape)
```

Python

```
Numeric columns used: ['Original Loan Amount', 'Original Loan To Value', 'SONYMA DPAL/CCAL Amount', 'Original Term', 'FIPS Code', 'Household Size', 'Loan
Categorical columns used: ['Bond Series', 'Loan Type', 'County', 'Number of Units', 'Property Type', 'Housing Type']
After dropna subset shape: (28528, 14)
```

num\_cols and cat\_cols: Machine learning algorithms require numeric input. We identify which columns are already numeric and which are categorical so we can handle them differently later.

loans\_df\_clean: Dates cannot be clustered directly, so we convert them into ordinal integers (days since year 1). This lets the model interpret time progression.

---

**03**

# Preprocessing

---



# Preprocessing Pipeline

```
# Identify column types
num_cols = df_subset.select_dtypes(include=['int64', 'float64']).columns
cat_cols = df_subset.select_dtypes(include=['object', 'category']).columns

# Check what columns are being used
print("Numeric columns:", num_cols.tolist())
print("Categorical columns:", cat_cols.tolist())

# Numeric pipeline
numeric_transformer = Pipeline(steps=[
    ... ('imputer', SimpleImputer(strategy='median')),
    ... ('scaler', StandardScaler())
])

# Categorical pipeline
categorical_transformer = Pipeline(steps=[
    ... ('imputer', SimpleImputer(strategy='constant', fill_value='MISSING')),
    ... ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
])

# Combine transformers
preprocessor = ColumnTransformer(
    ... transformers=[
    ... ... ('num', numeric_transformer, num_cols),
    ... ... ('cat', categorical_transformer, cat_cols)
    ... ])

# Fit-transform to create matrix for clustering
X = preprocessor.fit_transform(df_subset)
print("Feature matrix shape after preprocessing:", X.shape)
```

Preps data for clustering.

**numeric\_transformer:** For numeric columns, I first fill missing values with the median, then standardize them so all variables have comparable scales and no single feature dominates clustering

**categorical\_transformer:** For categorical variables, I replace missing values with "MISSING" and apply One-Hot Encoding, converting categories into binary columns the model can process

**column\_transformer:** This step applies the correct preprocessing automatically to the right columns, producing the final feature matrix used for clustering

Python

```
Numeric columns: ['Original Loan Amount', 'Original Loan To Value', 'SONYMA DPAL/CCAL Amount', 'Original Term', 'FIPS Code', 'Household Size', 'Loan Purchase Date', 'Purchase Year']
Categorical columns: ['Bond Series', 'Loan Type', 'County', 'Number of Units', 'Property Type', 'Housing Type']
Feature matrix shape after preprocessing: (28528, 213)
```

---

**04**

# **Dimensionality Reduction (PCA)**

---



# Dim reduction for visualization

```
pca = PCA(n_components=0.95, random_state=42)  # keep 95% variance
X_pca = pca.fit_transform(X)
print("PCA reduced shape:", X_pca.shape)
explained = pca.explained_variance_ratio_.sum()
print(f"PCA retained variance: {explained:.3f}")
```

Python

```
PCA reduced shape: (28528, 51)
```

```
PCA retained variance: 0.951
```

The dataset has many features after encoding. PCA reduces the number of dimensions while retaining 95% of the variance, so I can visualize clusters and speed up computation.

---

**05**

# **Gaussian Mixture Model (GMM) + Model Selection**



# What is Gaussian Mixture Model?

Unlike K-Means, which forces points into hard, spherical clusters, GMM is a probabilistic model. It assumes the data comes from overlapping bell-shaped distributions, which is more realistic for financial and demographic data where borrower profiles overlap. Instead of simply assigning each loan to one group, GMM calculates a probability of belonging to each cluster. Identifies clusters based on probability distributions rather than just distance like K-Means.

I test GMM models with 2–10 clusters and evaluate them using:

- BIC (Bayesian Information Criterion): lower is better, penalizes overly complex models
- AIC (Akaike Information Criterion): similar to BIC, lower is better
- Silhouette Score: measures how well-separated clusters are, higher is better

I selected the best number using BIC and Silhouette scores to avoid overfitting. This allowed me to identify meaningful borrower segments within the mortgage loan dataset.

To put it all together, GMM:

- Assigns probabilities of belonging to each cluster
  - Example: A borrower might be: 70% likely to belong to Cluster 0, 25% likely to belong to Cluster 2, 5% likely to belong to Cluster 1
- Allows flexible cluster shapes
  - K-Means creates spherical clusters, but real-world financial data is never really shaped that perfectly
- Probabilistic model
  - It estimates: each cluster's mean (center), covariance (shape and orientation), mixing weight (how large the cluster is)

# Why I chose Gaussian Mixture Model

---

My dataset contains financial and demographic attributes such as: original loan amount, purchase ratio, purchase date, original loan-to-value ratio, etc. Borrowers in New York are not all the same. There might be natural sub-groups. These borrower segments are not perfectly separated. They overlap. So a model that allows soft, probabilistic boundaries is more realistic than K-Means. Loan characteristics might correlate (higher income: larger loan: higher property value), producing stretched/elliptical clusters. GMM can show those relationships.

GMM helps answer questions like:

Is there a distinct theme in borrowers in the NY mortgage market?

Do certain counties or property types dominate specific clusters?

Can loan programs be better targeted based on cluster characteristics?

# Find optimal number of GMM component

```
n_components_range = range(2, 11) ... # try 2..10 clusters
bic_scores = []
aic_scores = []
sil_scores = []
gmm_models = {}

print("\nSearching best number of components for GMM...")
for n in n_components_range:
    gmm = GaussianMixture(n_components=n, covariance_type='full', random_state=42, n_init=5)
    gmm.fit(X)
    labels = gmm.predict(X)
    bic = gmm.bic(X)
    aic = gmm.aic(X)
    # silhouette requires at least 2 labels and less than n_samples clusters
    try:
        sil = silhouette_score(X, labels)
    except:
        sil = np.nan
    bic_scores.append(bic)
    aic_scores.append(aic)
    sil_scores.append(sil)
    gmm_models[n] = gmm
    print(f"n={n}: BIC={bic:.1f}, AIC={aic:.1f}, silhouette={sil:.4f}")

# Plot BIC/AIC and silhouette
plt.figure(figsize=(12,4))
plt.subplot(1,3,1)
plt.plot(list(n_components_range), bic_scores, marker='o')
plt.title("BIC vs n_components")
plt.xlabel("n_components")
plt.ylabel("BIC")

plt.subplot(1,3,2)
plt.plot(list(n_components_range), aic_scores, marker='o', color='orange')
plt.title("AIC vs n_components")
plt.xlabel("n_components")
plt.ylabel("AIC")
```

```
plt.subplot(1,3,3)
plt.plot(list(n_components_range), sil_scores, marker='o', color='green')
plt.title("Silhouette vs n_components")
plt.xlabel("n_components")
plt.ylabel("Silhouette")
plt.ylim(-0.1, 1)
plt.tight_layout()
plt.show()
```

```
# Choose best number of clusters: pick n with lowest BIC, but also consider silhouette.
best_n_bic = n_components_range[np.argmin(bic_scores)]
best_n_sil = n_components_range[np.nanargmax(sil_scores)]
print(f"Best by BIC: {best_n_bic}; Best by silhouette: {best_n_sil}")
```

```
# We'll pick the BIC-optimal one (you can change to best_n_sil)
chosen_n = best_n_bic
gmm_best = gmm_models[chosen_n]
labels = gmm_best.predict(X)
df_subset = df_subset.reset_index(drop=True)
df_subset['cluster'] = labels
```

**GaussianMixture():** creates GMM model with n clusters  
**gmm.fit(x):** learns the gaussian distributions from loan data  
**gmm.predict(x):** assigns each loan to the cluster where it has the highest probability of belonging  
**gmm.bic(x)** and **gmm.aic(x):** measures how well the models fits without being overly complex  
**silhouette\_score():** measures separation between clusters

# Output:

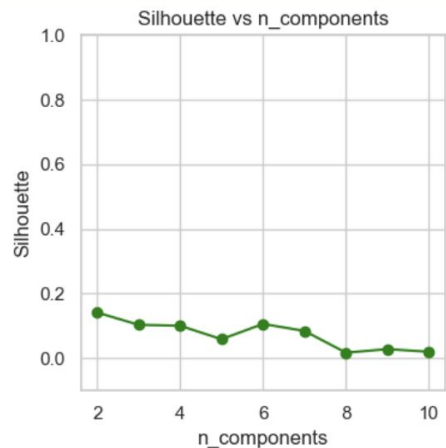
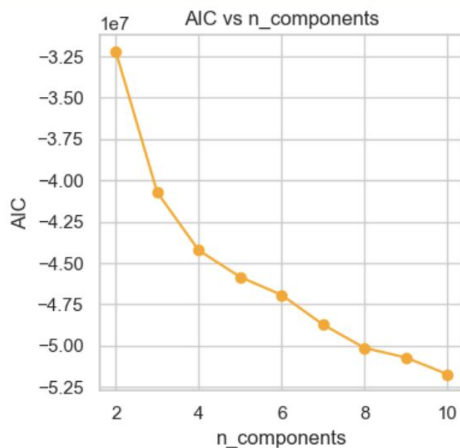
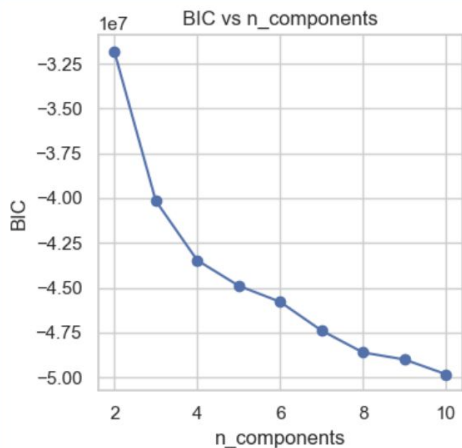
Searching best number of components for GMM...

```
n=2: BIC=-31811813.3, AIC=-32191785.1, silhouette=0.1411
n=3: BIC=-40154395.4, AIC=-40724357.3, silhouette=0.1030
n=4: BIC=-43469137.5, AIC=-44229089.4, silhouette=0.1001
n=5: BIC=-44891943.5, AIC=-45841885.5, silhouette=0.0584
n=6: BIC=-45788957.6, AIC=-46928889.6, silhouette=0.1057
n=7: BIC=-47392922.3, AIC=-48722844.3, silhouette=0.0841
n=8: BIC=-48614843.9, AIC=-50134756.0, silhouette=0.0168
n=9: BIC=-49007832.0, AIC=-50717734.1, silhouette=0.0275
n=10: BIC=-49827902.1, AIC=-51727794.2, silhouette=0.0197
```

Mortgage loans do not fall into perfectly separated groups. There are still underlying segments that GMM can model

The GMM assumption (loans come from a mix of borrower types) is realistic for mortgage markets. This makes sense because real borrowers overlap.

AIC/BIC tell which number of clusters best fits your data statistically. Silhouette tells how distinct the clusters are in feature space



Best by BIC: 10; Best by silhouette: 2

---

**06**

# **Cluster Summary and Interpretation**

---



# Cluster Summary:

```
print("\nCluster counts:")
print(df_subset['cluster'].value_counts().sort_index())

# For each cluster compute numeric means and categorical mode
cluster_summary_num = df_subset.groupby('cluster')[num_cols].mean().T
display(cluster_summary_num)

if len(cat_cols) > 0:
    ... cluster_summary_cat = {}
    ... for c in cat_cols:
    ...     cluster_summary_cat[c] = df_subset.groupby('cluster')[c].agg(lambda x: x.mode().iat[0] if len(x.mode()) > 0 else np.nan)
    ... cluster_summary_cat = pd.DataFrame(cluster_summary_cat)
    ... display(cluster_summary_cat)

# Show some example rows from each cluster
for k in sorted(df_subset['cluster'].unique()):
    ... print(f"\nExamples from cluster {k} (first 5 rows):")
    ... display(df_subset[df_subset['cluster']==k].head(5))
```

The dataset has many features after encoding. PCA reduces the number of dimensions while retaining 95% of the variance, so I can visualize clusters and speed up computation.

# Output:

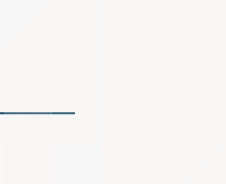
The dataset is not uniform. It contains distinct loan profiles. Common borrower types are dominant in clusters 2, 5, and 8. Smaller borrower types exist (clusters 6, 7). Possible cluster interpretation: first-time home buyers with these characteristics: lower income, smaller loans, government backed programs. OR Prime borrowers (low risk) with characteristics of a high credit score, strong income, etc. Each row represents a loan with features such as: Applicant income, Loan amount, Property value, Credit score, Interest rate, Loan term, Debt-to-income ratio. When I run GMM on the dataset: “Are there . natural groups of mortgage loans that share similar financial characteristics?” The clusters represent different types of mortgage loans, based on shared patterns in the features.

```
Cluster counts:
cluster
0    1219
1     3625
2     5898
3     2720
4     1154
5     5410
6     785
7      830
8     5703
9     1184
Name: count, dtype: int64
```

| cluster                 | 0             |               |               |               |               |               |               |               |               |               |
|-------------------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|
| Original Loan Amount    | 134367.670221 | 9764          |               |               |               |               |               |               |               |               |
| Original Loan To Value  | 0.899768      | 0.903463      | 0.799150      | 0.811865      | 0.909724      | 0.901279      | 0.847439      | 0.812884      | 0.901502      | 0.903928      |
| SONYMA DPAL/CCAL Amount | 2924.082811   | 4832.778944   | 6218.042162   | 8008.914271   | 4295.795389   | 5090.480345   | 9664.040094   | 5144.874429   | 5130.501167   | 2541.287193   |
| Original Term           | 360.000000    | 362.002759    | 360.000000    | 360.000000    | 357.452340    | 361.375231    | 480.000000    | 364.771084    | 360.000000    | 360.000000    |
| FIPS Code               | 36048.696473  | 36027.080000  | 36077.491353  | 36093.771324  | 36024.230503  | 36085.174861  | 36080.844586  | 36053.718072  | 36044.219008  | 36053.555743  |
| Household Size          | 1.767842      | 1.708690      | 2.069176      | 1.806250      | 1.909012      | 1.822921      | 2.411465      | 1.821687      | 1.606698      | 1.724662      |
| Loan Purchase Date      | 735053.557834 | 732677.275034 | 734844.773143 | 732548.594853 | 732514.964471 | 732527.816636 | 733773.982166 | 734075.425301 | 732494.649483 | 735451.186655 |
| Purchase Year           | 719163.000000 | 719163.000000 | 719163.000000 | 719163.000000 | 719163.000000 | 719163.000000 | 719163.000000 | 719163.000000 | 719163.000000 | 719163.000000 |

|         | Bond Series           | Loan Type    | County   | Number of Units | Property Type | Housing Type |
|---------|-----------------------|--------------|----------|-----------------|---------------|--------------|
| cluster |                       |              |          |                 |               |              |
| 0       | Series 38A/39/40      | Conventional | Monroe   | 1 Family        | Detached      | Existing     |
| 1       | Series 127/128/129    | Conventional | Monroe   | 1 Family        | Detached      | Existing     |
| 2       | Series 38D/46/47      | Conventional | Suffolk  | 1 Family        | Detached      | Existing     |
| 3       | Series 109/110        | Conventional | Suffolk  | 1 Family        | Detached      | Existing     |
| 4       | Series 116            | Conventional | Chemung  | 1 Family        | Detached      | Existing     |
| 5       | Series 113/114/115    | Conventional | Onondaga | 1 Family        | Detached      | Existing     |
| 6       | Series 38D/46/47      | Conventional | Suffolk  | 1 Family        | Detached      | Existing     |
| 7       | Series 160/161/162    | Conventional | Kings    | 1 Family        | Detached      | Existing     |
| 8       | Series 111/112        | Conventional | Monroe   | 1 Family        | Detached      | Existing     |
| 9       | 53rd Series (Taxable) | Conventional | Monroe   | 1 Family        | Detached      | Existing     |

---



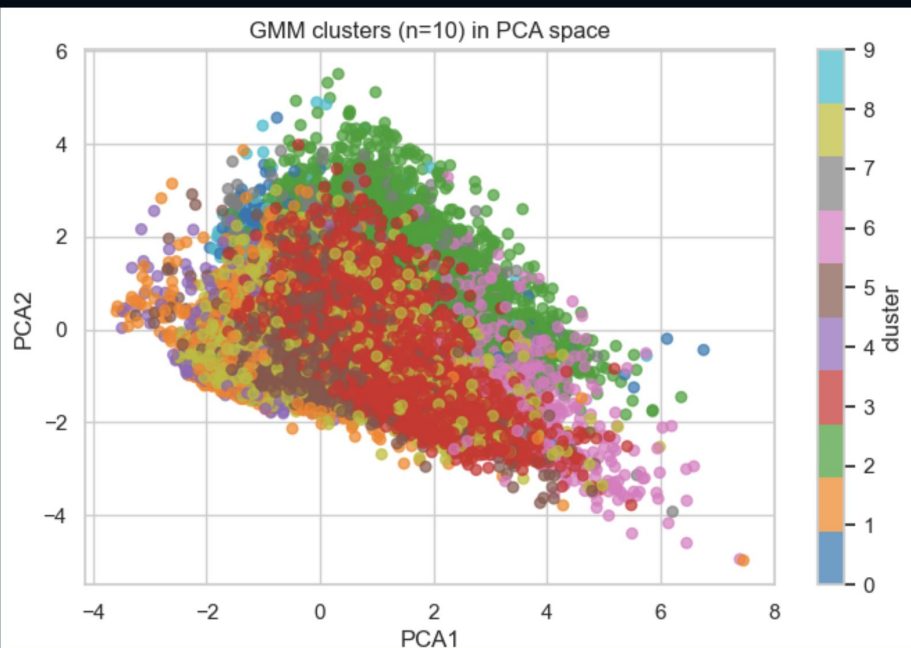
**07**

# **Visualize Clusters in 2d**



# Output:

```
plt.figure(figsize=(8,5))
plt.scatter(X_pca[:,0], X_pca[:,1], c=labels, cmap='tab10', s=30, alpha=0.7)
plt.title(f"GMM clusters (n={chosen_n}) in PCA space")
plt.xlabel("PCA1")
plt.ylabel("PCA2")
plt.colorbar(label='cluster')
plt.show()
```



I plotted the GMM cluster assignments in 2D using the first two principal components from PCA. This allowed me to visually inspect whether clusters are reasonably separated.

Each dot = one loan

Colors = which cluster GMM assigned it to

- The plot shows how loans group together based on similarity
- Shows that loans form multiple dense regions in feature space, meaning that borrowers and loan types are not all the same. There are meaningful subgroups in the mortgage market
- Overlap between clusters indicates that while segments exist, boundaries are not perfectly distinct, which is expected in real financial data

---

**08**

# **Save and Export**



## Wrote to new file

---

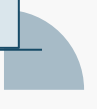
```
df_with_clusters = loans_df.copy()
# align indices if subset removed rows; here we kept

output_path = "sonyma_clusters_summary.csv"
df_subset.to_csv(output_path, index=False)
print("Wrote cluster summary to:", output_path)
```

```
Wrote cluster summary to: sonyma_clusters_summary.csv
```

---

# Summary

1. Cleaned dataset by detecting and converting currency, percentage, and date columns.
  2. After separating numeric and categorical features, I used a preprocessing pipeline that imputes missing values, scales numbers, and encodes categories.
  3. I reduced dimensionality with PCA and fitted a Gaussian Mixture Model, which identifies clusters based on probability distributions rather than just distance like K-Means.
  4. I selected the optimal number of clusters based on BIC, AIC, and Silhouette scores, and produced summaries to explain what characterizes each cluster.
- 

# 09 References

- Scikit-learn documentation:
  - GaussianMixture — <https://scikit-learn.org/stable/modules/mixture.html>
- t-SNE and Visualization: van der Maaten & Hinton, 2008
- Feature engineering best practices: Various blog posts and hands-on guides (Kaggle, Medium)
- Imbalanced datasets techniques: SMOTE, class weighting — scikit-learn-contrib imbalanced-learn

---



# Thanks!

Any questions?

CREDITS: This presentation template was created by [Slidesgo](#), and includes icons by [Flaticon](#), and infographics & images by [Freepik](#)

---

